AI Foundations I: Setup & Tools

A secure, ready-to-use AI research environment, configured in ten steps.

Nicholas Warren

Lead, Artificial Intelligence & Machine Learning program, DISSC

nick.warren@yale.edu

Session objectives

By the end of this session you will be able to:

- Set up a secure, containerized research environment with Docker and VS Code
- Navigate the VS Code interface and integrated terminal
- Understand how the Docker container, VS Code, and the AI service connect
- Verify that your environment is configured and ready to use

Why this matters

Research workflows in R involve repetitive, time-consuming tasks: cleaning data, debugging cryptic error messages, writing routine code, and iterating on visualizations. Claude Code can handle these so you can focus on the research questions.

- **Hours to minutes.** Tasks like reformatting data, writing plot code, or adding robust standard errors can be done by typing a single request.
- **Lower the barrier.** You don't need to memorize package syntax. Describe what you want in plain English and refine from there.
- **Stay in your editor.** No switching to a web browser. Claude works alongside your code and can read your files directly.

Setup

These instructions cover macOS and Windows. Follow only the slides for your platform.

1. Install Docker Desktop

Docker runs the workshop environment in an isolated container with R, Python, and Claude Code pre-installed. You won't need to install those tools yourself.

Download Docker Desktop from https://www.docker.com/products/docker-desktop/

1. Install Docker Desktop (macOS)

Open the downloaded `.dmg` file, drag Docker to your Applications folder, then launch it from Applications and complete the setup wizard.

After installation, make sure Docker Desktop is **running** (look for the whale icon in your menu bar at the top of the screen).

1. Install Docker Desktop (Windows)

Prerequisite: Docker Desktop requires **WSL 2** (Windows Subsystem for Linux). If you haven't installed it before, open **PowerShell as Administrator** (Start menu, type `PowerShell`, right-click on **Windows PowerShell** in the results, and select **Run as administrator**) and run:

```
wsl --install
```

Restart your computer when prompted. This only needs to be done once.

1. Install Docker Desktop (Windows, continued)

After the restart (or if WSL 2 is already installed), run the Docker Desktop installer and restart your computer if prompted.

After installation, make sure Docker Desktop is **running** (look for the whale icon in the system tray at the bottom-right of your screen).

2. Install VS Code (macOS)

Visual Studio Code (VS Code) is a free code editor made by Microsoft. We use it as the main working environment for the workshop.

- Download from https://code.visualstudio.com/
- Open the downloaded `.zip` file, then drag **Visual Studio Code** to your Applications folder

2. Install VS Code (Windows)

Visual Studio Code (VS Code) is a free code editor made by Microsoft. We use it as the main working environment for the workshop.

- Download from https://code.visualstudio.com/
- Run the downloaded installer and follow the prompts (keep the default options, including "Add to PATH")

3. Install the Dev Containers extension

Extensions are add-ons that give VS Code new features (similar to browser extensions or phone apps). The **Dev Containers** extension lets VS Code open projects inside Docker containers.

- Open VS Code
- Click the **Extensions** icon in the left sidebar (it looks like four small squares; or press `Cmd+Shift+X` on macOS / `Ctrl+Shift+X` on Windows)
- Search for **Dev Containers** by Microsoft and click **Install**

4. Download the sandbox (macOS)

- Go to https://github.com/DISSC-yale/claude-sandbox
- Click the green **Code** button, then click **Download ZIP**
- Find the downloaded `.zip` file (usually in your Downloads folder) and double-click it to unzip
- Move the extracted folder to a location you'll remember (e.g., your Desktop or Documents folder)

> You do not need Git installed. The ZIP download includes everything you need.

4. Download the sandbox (Windows)

- Go to https://github.com/DISSC-yale/claude-sandbox
- Click the green **Code** button, then click **Download ZIP**
- Find the downloaded `.zip` file (usually in your Downloads folder), right-click it, and select **Extract All**
- Move the extracted folder to a location you'll remember (e.g., your Desktop or Documents folder)

> You do not need Git installed. The ZIP download includes everything you need.

5. Add workshop files to the sandbox (macOS)

The sandbox starts with an empty `workspace/` folder. You need to copy the workshop files into it.

- Download the workshop files from https://github.com/DISSC-yale/ai-workshops (green **Code** button > **Download ZIP**, then double-click to unzip)
- Open the `ai-foundations-ii/vs-code-and-r/code/` folder
- **Important:** Some folders start with a dot (`.claude/` , `.vscode/`) and may be hidden. Press `Cmd+Shift+.` in Finder to show them.
- Select everything inside it and copy it into the sandbox's `workspace/` folder
- Also copy the `ai-foundations-ii/vs-code-and-r/prompts/` folder into `workspace/`

5. Add workshop files to the sandbox (Windows)

The sandbox starts with an empty `workspace/` folder. You need to copy the workshop files into it.

- Download the workshop files from https://github.com/DISSC-yale/ai-workshops (green **Code** button > **Download ZIP**, then right-click > **Extract All**)
- Open the `ai-foundations-ii\vs-code-and-r\code\` folder
- **Important:** Some folders start with a dot (`.claude\` , `.vscode\`) and may be hidden. In File Explorer, click **View** > **Show** > **Hidden items**.
- Select everything inside it and copy it into the sandbox's `workspace\` folder
- Also copy the `ai-foundations-ii\vs-code-and-r\prompts\` folder into `workspace\`

6. Request an API key

An **API key** is like a password that lets your computer connect to Claude. Email Nicholas Warren at DISSC (dissc@yale.edu) to request one. **Request this before the workshop;** you'll need the key in Step 7. This key will expire after the workshop. For long-term access, contact dissc@yale.edu.

7. Configure credentials (macOS)

Your computer needs to know your API key so it can pass it to Claude inside the container. We'll save it in a startup file that your Mac reads every time you open a terminal.

Open **Terminal** (`Cmd+Space`, type `Terminal`, press `Enter`). Then type the following command and press `Enter`:

```
nano ~/.zprofile
```

This opens a simple text editor called **nano**. (The file `~/.zprofile` is a hidden settings file in your home folder; `~/` means "my home folder".)

7. Configure credentials (macOS, continued)

In the nano editor, type (or paste) these three lines. Replace `TOKEN_HERE` with the API key you received in Step 6:

```
export AWS_REGION="us-east-1"
export AWS_BEARER_TOKEN_BEDROCK="TOKEN_HERE"
export CLAUDE_CODE_USE_BEDROCK=1
```

To save: press `Ctrl+O` (that's the **Control** key, not Command, plus the letter O), then press `Enter`. To exit: press `Ctrl+X`.

7. Configure credentials (macOS, continued)

Back in Terminal, run this command to apply your changes immediately:

```
source ~/.zprofile
```

> If your Mac uses `bash` instead of `zsh` (you can check by running `echo $SHELL`), use `~/.bash_profile` instead of `~/.zprofile` in all of the steps above.

> **Treat your API key like a password.** Do not share it or post it anywhere.

7. Configure credentials (Windows)

Your computer needs to know your API key so it can pass it to Claude inside the container. Windows stores these as **environment variables**, named settings that programs can read.

Click the Start menu, type `environment variables`, and select **Edit environment variables for your account**. Under **User variables**, click **New** to add each of these three entries (click **New**, type the name and value, click **OK**, then repeat):

Variable name	Value
`AWS_REGION`	`us-east-1`
`AWS_BEARER_TOKEN_BEDROCK`	your API key from Step 6
`CLAUDE_CODE_USE_BEDROCK`	`1`

7. Configure credentials (Windows, continued)

Click **OK** to close the dialog. Then **restart** any open terminals or applications so they pick up the new settings.

> **Treat your API key like a password.** Do not share it or post it anywhere.

8. Open the workshop in a container

- Open **VS Code**
- Go to `File > Open Folder` and select the sandbox folder you extracted in Step 4
- When VS Code prompts **"Reopen in Container?"**, click **Yes**

> If the prompt doesn't appear, press `Cmd+Shift+P` (macOS) or `Ctrl+Shift+P` (Windows), type `Reopen in Container`, and select it.

8. Open the workshop in a container (continued)

The first build takes **15–25 minutes** because it installs and compiles R packages. Subsequent opens are fast (a few seconds). A log will appear showing setup progress; you can let it run in the background.

Once it finishes, close that log tab and open a fresh terminal inside VS Code with `Ctrl+` (that's the backtick key, usually above Tab).

9. Install VS Code extensions

You're now working inside the container. R and Python are already installed, but you need two VS Code extensions that add R and Quarto support:

- Click the **Extensions** icon in the left sidebar (four small squares; or `Cmd+Shift+X` / `Ctrl+Shift+X`)
- Search for **R** by REditorSupport and click **Install**
- Search for **Quarto** by Quarto and click **Install**

10. Verify your setup

In the container's terminal, start Claude Code:

```
claude
```

Then ask a test question: `What is 2 + 2?`

You should see Claude respond with an answer. Type `/exit` to quit Claude Code.

> **If it doesn't work:** Make sure Docker Desktop is running and your credentials are saved (Step 7).

Orientation

Now that your environment is set up, let's understand the tools you just installed.

What is Docker?

Docker creates a self-contained workspace on your computer, like a virtual computer within your computer. This workspace is called a **container**.

Why use it?

- **No installation headaches.** R, Python, and Claude Code are pre-configured and ready to go.
- **Consistent environment.** Everyone runs the same container, so there are no "it works on my machine but not yours" problems.
- **Safe to experiment.** The container is isolated from the rest of your computer. Claude can only access files in the `workspace/` folder and cannot touch your personal files, email, or anything else on your machine.

What is the terminal?

The **terminal** is a text-based interface for your computer. Instead of clicking icons, you type commands. Think of it like the R console, but for your entire machine.

Concept	What it means
Shell	The program that interprets your commands (you don't need to worry about which one you have)
Command	A text instruction like `ls` (list files) or `cd` (change folder)
Working directory	The folder your terminal is currently "in"

Essential terminal commands

These are the commands you'll use most often:

Command	What it does	Example
`ls`	List files in the current folder	`ls data/`
`cd`	Change directory (move to another folder)	`cd ~/Documents`
`pwd`	Print the current working directory	`pwd`
`cat`	Print the contents of a file	`cat data.csv`
`claude`	Start Claude Code	`claude`

> You don't need to memorize these. Claude Code can run them for you.

What is VS Code?

Visual Studio Code (VS Code) is a free code editor made by Microsoft. Think of it as a modern alternative to RStudio that works with many languages.

Key differences from RStudio:

- VS Code supports **any programming language** (R, Python, Stata, etc.) through add-ons called **extensions**
- It has a built-in **terminal** where you can type commands (like the one you used during setup)
- It is the most widely used code editor in the world, with a large library of extensions

Navigating the VS Code interface

Area	Location	What it does
Activity Bar	Far left (icons)	Switch between Explorer, Search, Extensions
Side Bar	Left panel	File tree, search results, or extensions
Editor	Center	View and edit files. Run code with `Cmd/Ctrl + Enter`
Panel	Bottom	Terminal and R Terminal. Toggle with `Ctrl + ` `

Key actions in VS Code

Action	How
Find your files	Click the top Activity Bar icon to open the **Explorer**
Open a terminal	`Ctrl + ` ` or `Terminal > New Terminal`
Install extensions	`Cmd/Ctrl + Shift + X` to open the Extensions view
Command Palette	`Cmd/Ctrl + Shift + P` (universal search for any command or setting)

VS Code for RStudio users

RStudio	VS Code equivalent
Console	**R Terminal** (bottom Panel)
Source editor	**Editor** (center). Run lines with `Cmd/Ctrl + Enter`
Files pane	**Explorer** (top icon in Activity Bar)
Environment pane	**R: Workspace** (R icon in Activity Bar)
Plots pane	Inline in the notebook (`.qmd` files, similar to R Markdown)
Help	Hover over a function, or `?fn` in R Terminal

How it works

In this workshop you use **VS Code** as your editor and **Claude Code** as your AI assistant, all running inside a **Docker container**. Claude Code is a tool made by **Anthropic** (the company behind Claude) that lets you interact with Claude directly from the terminal.

When you type a message to Claude Code, it is sent to **Claude Opus 4.6** (the most capable Claude model) through **AWS Bedrock**, Amazon's AI service, running on Yale's AWS account.

Data security

- **Your data is not used for training.** AWS Bedrock does not use your prompts or outputs to train any models.
- **Your data is not shared.** Model providers (including Anthropic) do not have access to your inputs, outputs, or logs.
- **Enterprise compliance.** AWS Bedrock is HIPAA eligible, SOC and ISO certified, and GDPR compliant.

In short: you get the full power of Claude without your research data leaving Yale's AWS environment or being used to train future models.

Key concepts

Tool	What it is
Docker	Creates the self-contained workshop environment with all tools pre-installed.
VS Code	A free code editor from Microsoft. Your main working environment.
Dev Containers	A VS Code add-on (extension) that connects VS Code to Docker containers.
AWS Bedrock	Amazon's AI service. Your requests to Claude go through Yale's AWS account.
API key	A password-like key that proves you're authorized to use Claude.

One project per sandbox

Each sandbox is meant for a single project. Claude operates on the entire `workspace/` folder, so mixing unrelated files risks unintended changes across projects.

To start a new project, **duplicate the entire sandbox folder** and rename the copy (e.g., `my-project-name`). Each copy runs its own isolated container with its own `workspace/`.

Using the sandbox without R or Python

This workshop uses R, but the sandbox also works as a general-purpose Claude Code environment. If you want a lighter container for non-R projects, see the **"Customizing the Container"** section of the sandbox's `README.md` for instructions on removing R and Python. This shrinks the image and speeds up the first build.